

Predicting 30-Day Hospital Readmission

Using MIMIC-III data to estimate, ahead of time, which patients are likely to come back within 30 days.

Why Try to Predict This?

Hard to see by eye

A doctor can't track every risk factor for every patient by hand. The pattern is real, but it's hidden in the data.

Costly to get wrong

A patient coming back usually means something wasn't fully treated the first time - harder on the patient, and expensive to repeat.

Something we can test

MIMIC-III lets us check whether this pattern is strong enough to predict in advance, not just describe after the fact.

Setting Up the Problem

The Label

For every hospital stay, we look at the patient's next admission, if there is one. If it starts within 30 days of this discharge, we mark this row as a readmission (1). Otherwise, 0.

One row in our table = one hospital admission, not one patient - the same patient can appear more than once.

The Cohort

Data comes from MIMIC-III, pulled live through Google BigQuery.

We leave newborn admissions out of the cohort, since they behave very differently from adult stays and would skew the pattern.

FEATURES

What We Feed the Model

`admission_type`

Emergency, urgent, or planned. Emergency patients tend to be less medically stable going in.

`prior_admission_count`

How many times this patient has been admitted before. More visits often means a chronic condition.

`primary_diagnosis`

The main reason for this stay. Some conditions come back more easily than others.

`lab_abnormal_pct`

How much of this stay's lab work came back outside the normal range.

`age, gender`

Basic facts, included as a check - not assumed to matter until the model says so.

FINDING

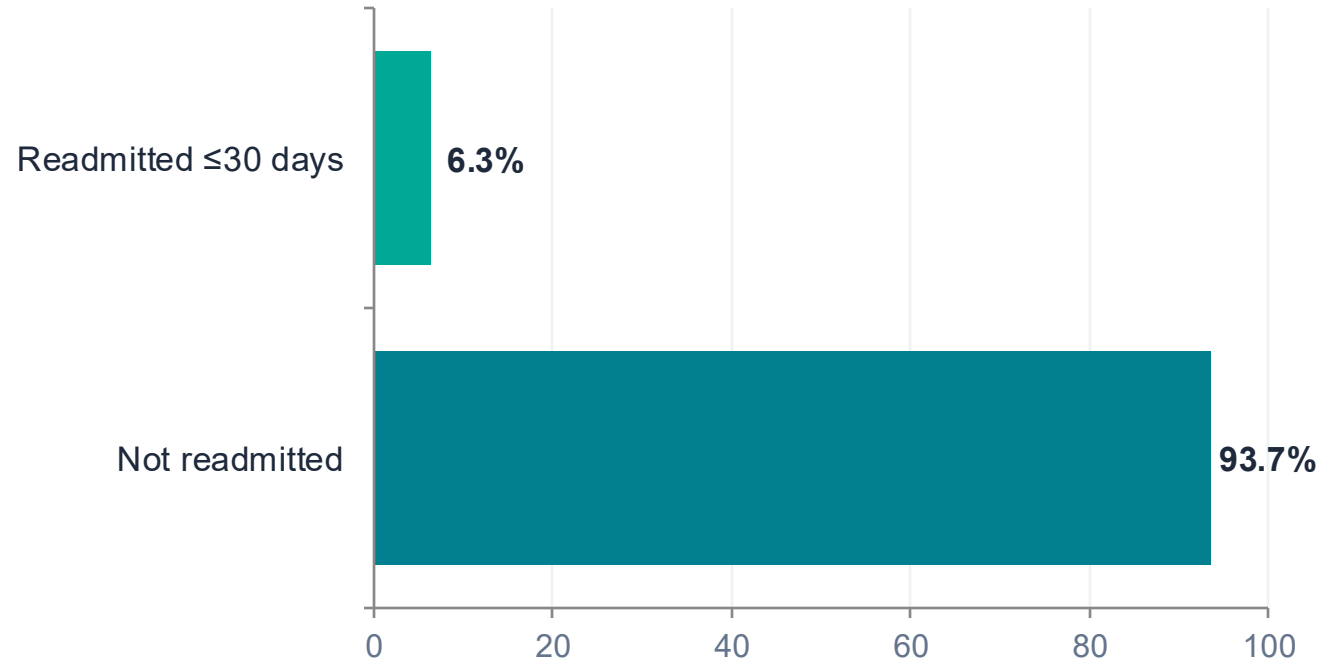
MIMIC-III shifts the recorded age of anyone over 89, to protect their privacy. That can make raw data show ages of 200+ years.

We cap age at 90 in our query, so the model isn't confused by this quirk.

Text categories like `admission_type` and `primary_diagnosis` get turned into separate yes/no columns later — more on that shortly.

Readmission Is Rare

Share of Admissions



6.3%

of admissions were followed by a readmission within 30 days.

- Most rows are “no” - this is called class imbalance.
- Always guessing “no” would score ~94% while being useless.
- So we won't judge the model by plain accuracy.

Getting the Data Model-Ready

1. One-hot encode

Turn each text category into its own yes/no column. A model can only work with numbers, not words.

2. Split the data

80% for training, 20% for testing — keeping the same 6.3% readmission rate in both, so the test is a fair check.

3. Scale the numbers

Put every number on a similar scale, so a big number like total labs doesn't drown out a small one like age.

Start Simple: Logistic Regression

Logistic regression guesses a probability of readmission, from 0 to 1, by adding up weighted features. Each feature gets a weight, and the model combines them.

It's simple and easy to check afterward - a fair baseline. If a neural network can't beat it, the extra complexity isn't worth it.

FINDING

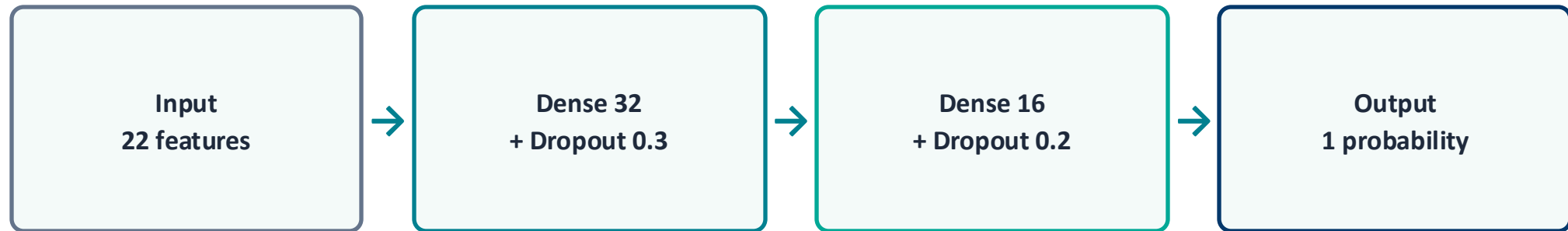
0.657

AUROC

Meaningfully better than a coin flip (0.5). This is the score the neural network has to beat.

class_weight='balanced' tells the model to pay more attention to the rare readmit cases, instead of mostly learning from the common "no" cases.

A Small Neural Network, By Design



Why 32 → 16 nodes: with only ~22 columns after encoding, a much bigger network would just memorize the training rows instead of learning a pattern that generalizes.
Why dropout 0.3, then 0.2: dropout randomly turns off nodes during training so the model can't lean on just one - set higher near the input, since that's where overfitting risk is greatest with so few readmit cases to learn from.

Why We Don't Trust Accuracy Here

Accuracy

Misleading here. Always guessing “no” scores about 94% while catching zero real readmissions.

AUROC

How well the model ranks readmitted patients above everyone else. 0.5 = random guessing, 1.0 = perfect.

AUPRC

A stricter score, focused only on the rare readmit group. Closer to the question a care team actually asks.

RESULTS

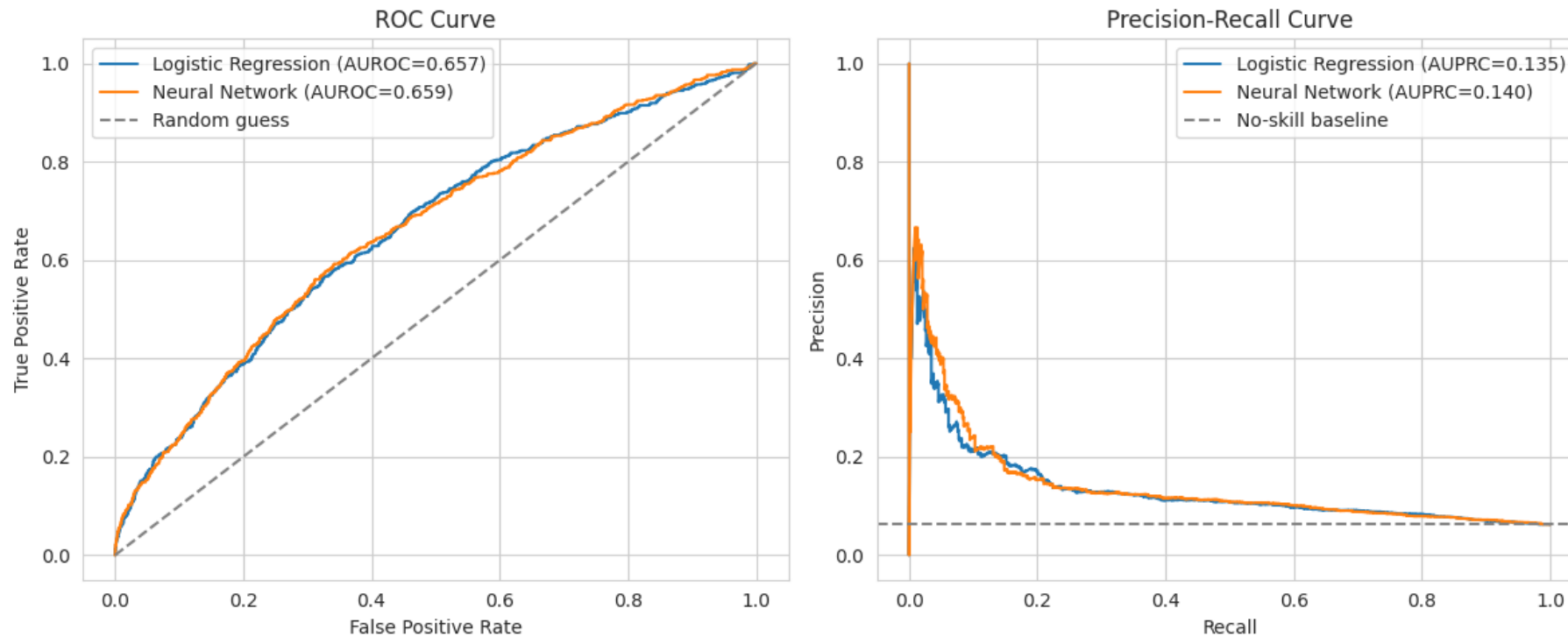
Logistic Regression vs. Neural Network

Both models land in almost the same place.

Model	AUROC	AUPRC
Logistic Regression	0.657	0.135
Neural Network	0.659	0.140

For a small, mostly-tabular dataset like this one, a simple model can do nearly as well as a neural network. That's a fair result to report honestly, not something to explain away.

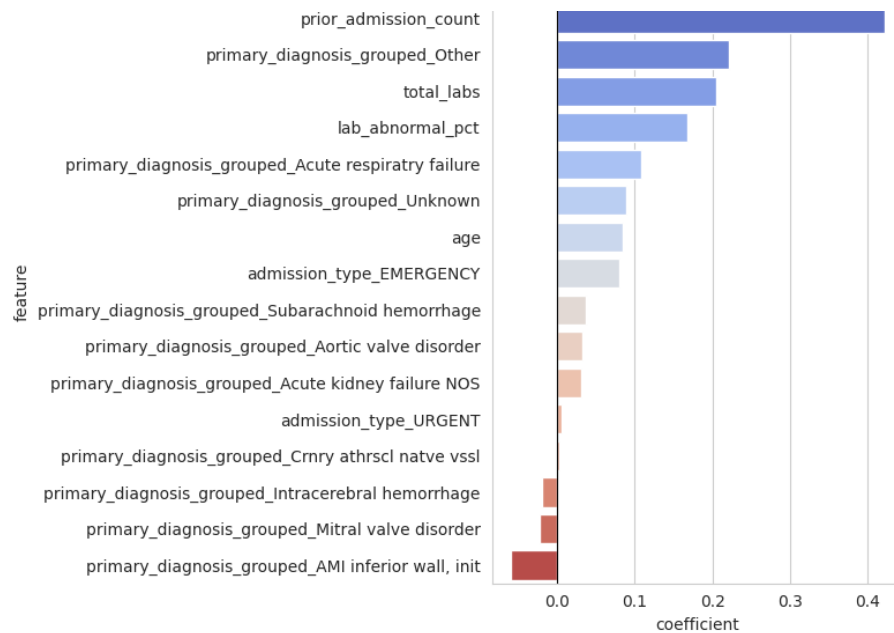
Recall vs. Accuracy at the Cutoff



At a 0.5 cutoff: Logistic Regression catches 55% of true readmissions and stays 67% accurate overall. The neural network catches more - 66% - but its accuracy drops to 57%, since it flags more false alarms. Which model is “better” depends on what costs more: missing a real readmission, or an unnecessary follow-up call.

INTERPRETATION

What Actually Predicts Readmission?



prior_admission_count is the strongest predictor, by a clear margin.

FINDING

A few diagnoses — kidney failure, intracerebral hemorrhage — seem to LOWER predicted risk.

But these also carry higher in-hospital mortality. A patient who doesn't survive can't be readmitted — so this is probably about survival, not a safer outcome.

What We Learned

Takeaways

- Readmission is somewhat predictable, but not strongly — both models land around 0.66 AUROC.
- A simple model did almost as well as a neural network here.
- A patient's admission history is the clearest signal we found.
- Where to set the risk cutoff is a choice for the hospital, not the model.

Limitations

- We didn't use any of the discharge note text yet.
- Some features come from the same admission we're predicting on — a real deployment would need to check timing more carefully.
- The mortality mix-up noted earlier isn't separately handled.
- No tuning or cross-validation — kept simple on purpose for this tutorial.

Try It Yourself

- 1 Get PhysioNet-credentialed access to MIMIC-III, and load it into Google BigQuery.
- 2 Run the feature-engineering SQL query to build the admission-level dataset.
- 3 Install scikit-learn and TensorFlow.
- 4 Run the notebook top to bottom - no manual steps needed in between.

Full commented code, with the complete notebook, is linked in this deck's title-slide notes.